

Angular 22 consolida el paradigma Zoneless y la arquitectura Signals-First, marcando un punto de inflexión en el rendimiento, la ergonomía de desarrollo y la integración nativa con herramientas de IA.

Mauro Bernal

Módulo 1: Fundamentos y Nueva Detección de Cambios

- **OnPush por defecto:** Todos los componentes generados en Angular 22 adoptan `ChangeDetectionStrategy.OnPush`. Se eliminan los chequeos globales redundantes de `zone.js` en favor de un rastreo síncrono y de grano fino con Signals.
- **Consolidador Zoneless:** Desarrollo nativo sin dependencia obligatoria de `zone.js` para bundles significativamente más livianos y un rendimiento superior.
- **Integración de Vitest:** El CLI de Angular 22 adopta Vitest como motor predeterminado de pruebas unitarias, reemplazando a Karma/Jasmine.

Mauro Bernal

TypeScript

```
import { Component, signal, ChangeDetectionStrategy } from '@angular/core';

@Component({
  selector: 'app-contador',
  standalone: true,
  // ChangeDetectionStrategy.OnPush es implícito a partir de v22
  template: `
    <p>Conteo: {{ count() }}</p>
    <button (click)="incrementar()">Incrementar</button>
  `
})
export class ContadorComponent {
  count = signal(0);
  incrementar() {
    this.count.update(c => c + 1);
  }
}
```

Módulo 2: APIs Asíncronas y Manejo de Estado (resource y httpResource)

- **resource() y httpResource()** Estables: Reemplazan el código repetitivo de peticiones HTTP manuales con cancelación automática de peticiones en vuelo, reactividad basada en Signals e hidratación fluida.

campusMVP

TypeScript

```
import { Component, signal, httpResource } from '@angular/core';

@Component({
```

```

selector: 'app-usuarios',
standalone: true,
template: `
  @if (usuarios.isLoading()) {
    <p>Cargando datos...</p>
  } @else if (usuarios.error()) {
    <p>Error al obtener usuarios.</p>
  } @else {
    <ul>
      @for (user of usuarios.value(); track user.id) {
        <li>{{ user.name }}</li>
      }
    </ul>
  }
`
})
export class UsuariosComponent {
  searchQuery = signal('admin');

  // Reactividad automática al modificar searchQuery()
  usuarios = httpResource<User[]>(() => `/api/users?role=$
{this.searchQuery()}`);
}

```

Módulo 3: Formularios de Nueva Generación (Signal Forms)

- @angular/forms/signals Estable: Formularios reactivos contruidos 100% sobre Signals.
Mauro Bernal
- Debounce Nativo y Soporte Zod/Valibot: Reglas integradas como debounce(field, delay) sin necesidad de operadores de RxJS, soporte nativo para esquemas de validación tipados y validadores como minDate() y maxDate().

TypeScript

```

import { Component, signal } from '@angular/core';
import { signalForm, formField, validators } from '@angular/forms/signals';

@Component({
  selector: 'app-login',
  standalone: true,
  template: `
    <form [formGroup]="form">
      <input [formField]="form.controls.email" placeholder="Correo" />
      @if (form.controls.email.errors()?.required) {
        <span>Campo obligatorio</span>
      }
    </form>
  `
})
export class LoginComponent {
  form = signalForm({
    email: formField('', {
      validators: [validators.required, validators.email],
      debounce: 300 // Debouncing síncrono nativo
    })
  })
}

```

```
});  
}
```

Módulo 4: Enrutamiento Avanzado y Plantillas Modernas

- Herencia Automática de Parámetros: La estrategia `paramsInheritanceStrategy` ahora es `'always'` por defecto; las rutas hijas heredan automáticamente los parámetros del padre sin configuraciones extra.

YouTube

- Sintaxis Spread y Funciones Flecha Inline: Las plantillas aceptan propagación de objetos `[style]="...estilos"` y lambdas de un solo uso en asignaciones de eventos.
- Límites de Error con `<ng-boundary>`: Aislamiento de fallos de renderizado en fragmentos críticos del DOM para evitar caídas globales de la interfaz.

Módulo 5: Accesibilidad y Web Agéntica (IA)

- Angular Aria (Estable): Directivas y patrones sin estilos (12 patrones de interfaz accesibles) integrados nativamente con Signal Forms para garantizar estándares WCAG.

Mauro Bernal

- Ecosistema Agéntico (MCP y WebMCP): Herramientas para interoperar con agentes de Inteligencia Artificial mediante el protocolo MCP (Model Context Protocol), incluyendo bucles de autocorrección en compilación y exposición de herramientas estructuradas directamente en el navegador.

Comando de Migración

Para actualizar proyectos existentes desde Angular 21 a Angular 22:

www.yeou.dev

Bash

```
ng update @angular/core@22 @angular/cli@22
```

(La migración automática conservará la compatibilidad añadiendo

`ChangeDetectionStrategy.Eager` en tus componentes legacy para no romper lógica preexistente).

En este tutorial construiremos una aplicación CRUD completa de Gestión de Productos utilizando las APIs estables de Angular 22: `httpResource` para la gestión asíncrona de datos y `Signal Forms (@angular/forms/signals)` para el manejo del formulario reactivo con validación tipada.

Paso 1: Definir la Interfaz de Datos

Crea el modelo `product.model.ts` que represente la estructura del recurso en la API REST:

TypeScript

```
// src/app/models/product.model.ts
export interface Product {
  id: number;
  title: string;
  price: number;
  category: string;
  description: string;
}

export type CreateProductDto = Omit<Product, 'id'>;
```

Paso 2: Crear el Servicio HTTP con Signal Forms y `httpResource`

A diferencia de los métodos basados en RxJS (`HttpClient.get`), `httpResource` devuelve un objeto reactivo que gestiona automáticamente los estados `value()`, `isLoading()` y `error()`.

Crea el servicio `product.service.ts`:

TypeScript

```
// src/app/services/product.service.ts
import { Injectable, signal, httpResource, inject } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Product, CreateProductDto } from '../models/product.model';

@Injectable({
  providedIn: 'root'
})
export class ProductService {
  private http = inject(HttpClient);
  private readonly apiUrl = 'https://fakestoreapi.com/products';

  // Signal para controlar la categoría filtrada activa
  selectedCategory = signal<string>('');

  // httpResource reacciona automáticamente al cambio en selectedCategory()
  productsResource = httpResource<Product[]>(() => {
    const category = this.selectedCategory();
    return category
      ? `${this.apiUrl}/category/${category}`
      : this.apiUrl;
  });

  // Método para refrescar manualmente el recurso
  reload() {
    this.productsResource.reload();
  }

  // Operaciones de escritura (Mutaciones)
  createProduct(product: CreateProductDto) {
    return this.http.post<Product>(this.apiUrl, product);
  }
}
```

```

    }

    updateProduct(id: number, product: Partial<CreateProductDto>) {
      return this.http.put<Product>(`${this.apiUrl}/${id}`, product);
    }

    deleteProduct(id: number) {
      return this.http.delete<Product>(`${this.apiUrl}/${id}`);
    }
  }
}

```

Paso 3: Construir la Vista Principal (Tabla + Filtro + Estados de Carga)

Aprovecharemos la sintaxis de plantillas con bloques `@if`, `@for` y los estados nativos expuestos por `httpResource`.

Crea el componente `product-list.component.ts`:

TypeScript

```

// src/app/components/product-list/product-list.component.ts
import { Component, inject, signal } from '@angular/core';
import { ProductService } from '../../../services/product.service';
import { Product } from '../../../models/product.model';
import { ProductFormComponent } from '../product-form/product-form.component';

@Component({
  selector: 'app-product-list',
  standalone: true,
  imports: [ProductFormComponent],
  template: `
    <div class="container my-4">
      <div class="d-flex justify-content-between align-items-center mb-4">
        <h2>Gestión de Productos (Angular 22)</h2>
        <button class="btn btn-primary" (click)="openCreateModal()">
          + Nuevo Producto
        </button>
      </div>

      <!-- Filtro por categoría -->
      <div class="mb-3 w-25">
        <label class="form-label font-semibold">Filtrar por categoría:</label>
        <select
          class="form-select"
          (change)="onCategoryChange($event)"
        >
          <option value="">Todas</option>
          <option value="electronics">Electrónica</option>
          <option value="jewelery">Joyería</option>
          <option value="men's clothing">Ropa de Hombre</option>
        </select>
      </div>

      <!-- Manejo de Estados con httpResource -->
      @if (productService.productsResource.isLoading()) {
        <div class="alert alert-info d-flex align-items-center gap-2">
          <div class="spinner-border spinner-border-sm" role="status"></div>
          <span>Cargando productos con httpResource...</span>
        </div>
      }
    </div>
  `
})

```

```

    } @else if (productService.productsResource.error()) {
        <div class="alert alert-danger">
            Ocurrió un error al cargar la lista:
    {{ productService.productsResource.error() }}
        </div>
    } @else {
        <table class="table table-hover border">
            <thead class="table-light">
                <tr>
                    <th>ID</th>
                    <th>Título</th>
                    <th>Precio</th>
                    <th>Categoría</th>
                    <th class="text-end">Acciones</th>
                </tr>
            </thead>
            <tbody>
                @for (product of productService.productsResource.value(); track
product.id) {
                    <tr>
                        <td>{{ product.id }}</td>
                        <td class="fw-medium">{{ product.title }}</td>
                        <td>\${{ product.price }}</td>
                        <td><span class="badge bg-secondary">{{ product.category
}}</span></td>
                        <td class="text-end">
                            <button
                                class="btn btn-sm btn-outline-warning me-2"
                                (click)="openEditModal(product)"
                            >
                                Editar
                            </button>
                            <button
                                class="btn btn-sm btn-outline-danger"
                                (click)="deleteProduct(product.id)"
                            >
                                Eliminar
                            </button>
                        </td>
                    </tr>
                } @empty {
                    <tr>
                        <td colspan="5" class="text-center text-muted">No hay productos
disponibles.</td>
                    </tr>
                }
            </tbody>
        </table>
    }

    <!-- Formulario Modal / Drawer -->
    @if (showFormModal()) {
        <app-product-form
            [productToEdit]="selectedProduct()"
            (closeModal)="closeFormModal()"
        />
    }
</div>

```

```

})
export class ProductListComponent {
  productService = inject(ProductService);

  showFormModal = signal(false);
  selectedProduct = signal<Product | null>(null);

  onCategoryChange(event: Event) {
    const value = (event.target as HTMLSelectElement).value;
    this.productService.selectedCategory.set(value);
  }

  openCreateModal() {
    this.selectedProduct.set(null);
    this.showFormModal.set(true);
  }

  openEditModal(product: Product) {
    this.selectedProduct.set(product);
    this.showFormModal.set(true);
  }

  closeFormModal() {
    this.showFormModal.set(false);
  }

  deleteProduct(id: number) {
    if (confirm('¿Estás seguro de eliminar este producto?')) {
      this.productService.deleteProduct(id).subscribe({
        next: () => {
          alert('Producto eliminado correctamente');
          this.productService.reload();
        }
      });
    }
  }
}

```

Paso 4: Crear el Formulario con Signal Forms

Con la API de Signal Forms (`@angular/forms/signals`), el estado completo del formulario y la validación residen directamente en Signals, eliminando la necesidad de `FormGroup` tradicional e incluyendo funciones como `debounce` nativo.

Crea el componente `product-form.component.ts`:

TypeScript

```

// src/app/components/product-form/product-form.component.ts
import { Component, inject, input, output, OnInit } from '@angular/core';
import { signalForm, formField, validators } from '@angular/forms/signals';
import { ProductService } from '../../../services/product.service';
import { Product } from '../../../models/product.model';

@Component({
  selector: 'app-product-form',
  standalone: true,
  template: `

```

```

<div class="modal d-block bg-dark bg-opacity-50" tabindex="-1">
  <div class="modal-dialog modal-dialog-centered">
    <div class="modal-content">
      <div class="modal-header">
        <h5 class="modal-title">
          {{ productToEdit() ? 'Editar Producto' : 'Nuevo Producto' }}
        </h5>
        <button type="button" class="btn-close"
(click)="closeModal.emit()"></button>
      </div>

      <form (submit)="onSubmit($event)">
        <div class="modal-body">

          <!-- Campo Título con Debounce Nativo -->
          <div class="mb-3">
            <label class="form-label">Título</label>
            <input
              type="text"
              class="form-control"
              [formField]="productForm.controls.title"
              placeholder="Ej. Teclado Mecánico"
            />
            @if (productForm.controls.title.errors()?.required) {
obligatorio.</small>
            }
          </div>

          <!-- Campo Precio -->
          <div class="mb-3">
            <label class="form-label">Precio</label>
            <input
              type="number"
              class="form-control"
              [formField]="productForm.controls.price"
            />
            @if (productForm.controls.price.errors()?.min) {
mayor a 0.</small>
            }
          </div>

          <!-- Campo Categoría -->
          <div class="mb-3">
            <label class="form-label">Categoría</label>
            <select class="form-select"
[formField]="productForm.controls.category">
              <option value="">Selecciona una opción</option>
              <option value="electronics">Electrónica</option>
              <option value="jewelery">Joyería</option>
              <option value="men's clothing">Ropa de Hombre</option>
            </select>
          </div>

          <!-- Campo Descripción -->
          <div class="mb-3">
            <label class="form-label">Descripción</label>
            <textarea

```



```

        class="form-control"
        rows="3"
        [formField]="productForm.controls.description"
    ></textarea>
</div>

</div>

<div class="modal-footer">
    <button
        type="button"
        class="btn btn-secondary"
        (click)="closeModal.emit()"
    >
        Cancelar
    </button>
    <button
        type="submit"
        class="btn btn-success"
        [disabled]="!productForm.valid()"
    >
        Guardar
    </button>
</div>
</form>
</div>
</div>
</div>
,
})
export class ProductFormComponent implements OnInit {
    private productService = inject(ProductService);

    // Entradas y Salidas con la sintaxis de Signal I/O
    productToEdit = input<Product | null>(null);
    closeModal = output<void>();

    // Definición del Signal Form con reglas y debounce nativo
    productForm = signalForm({
        title: formField('', {
            validators: [validators.required],
            debounce: 250 // Aplica retardo síncrono en la actualización del Signal
        }),
        price: formField(0, {
            validators: [validators.required, validators.min(0.01)]
        }),
        category: formField('electronics', {
            validators: [validators.required]
        }),
        description: formField('')
    });

    ngOnInit() {
        // Si viene un producto para editar, cargamos los valores en el formulario
        const product = this.productToEdit();
        if (product) {
            this.productForm.setValue({
                title: product.title,
                price: product.price,
            });
        }
    }
}

```

```

        category: product.category,
        description: product.description
    });
}
}

onSubmit(event: Event) {
    event.preventDefault();

    if (!this.productForm.valid()) return;

    const formValues = this.productForm.value();
    const productData = {
        title: formValues.title,
        price: Number(formValues.price),
        category: formValues.category,
        description: formValues.description
    };

    const currentProduct = this.productToEdit();

    if (currentProduct) {
        // Edición
        this.productService.updateProduct(currentProduct.id,
productData).subscribe({
            next: () => {
                this.productService.reload();
                this.closeModal.emit();
            }
        });
    } else {
        // Creación
        this.productService.createProduct(productData).subscribe({
            next: () => {
                this.productService.reload();
                this.closeModal.emit();
            }
        });
    }
}
}

```

Paso 5: Configurar la Aplicación Root

En el archivo `app.config.ts`, debemos registrar el cliente HTTP con soporte para peticiones integradas en el entorno Zoneless:

TypeScript

```

// src/app/app.config.ts
import { ApplicationConfig, provideZoneChangeDetection } from '@angular/core';
import { provideHttpClient, withFetch } from '@angular/common/http';

export const appConfig: ApplicationConfig = {
    providers: [
        // Opción Zoneless o detección optimizada según tu entorno
        provideZoneChangeDetection({ eventCoalescing: true }),
        provideHttpClient(withFetch())
    ]
}

```

```
};
```

Puntos Clave de la Arquitectura en Angular 22:

1. `httpResource` reactivo: Al cambiar `selectedCategory()`, `httpResource` detecta la mutación del signal y vuelve a ejecutar la petición automáticamente sin necesidad de `switchMap` de RxJS.
2. Signal Forms: Todo el ciclo de vida del formulario (`valid()`, `errors()`, `value()`) expone datos a través de Signals nativos, lo que garantiza actualizaciones del DOM ultrarrápidas y sin comprobaciones innecesarias de detección de cambios.
3. Manejo de estados sin boilerplate: Las propiedades `.isLoading()` y `.error()` de `httpResource` evitan tener que declarar manualmente flags booleanos de carga o captura de excepciones HTTP.

Incluye una página de detalles.

Para completar el flujo CRUD/Navegación en la aplicación, añadiremos la vista de detalles del producto.

Utilizaremos la función `injectParams()` de la API de enrutamiento de Angular para leer el parámetro de la URL como un Signal reactivo y consumirlo directamente dentro de un `httpResource` parametrizado.

Paso 1: Crear el Componente de Detalles (`product-detail.component.ts`)

Este componente obtiene la ID desde la ruta, realiza la petición automática mediante `httpResource` y muestra la información del producto de forma aislada.

TypeScript

```
// src/app/components/product-detail/product-detail.component.ts
import { Component, inject, httpResource } from '@angular/core';
import { RouterLink, injectParams } from '@angular/router';
import { Product } from '../../../models/product.model';

@Component({
  selector: 'app-product-detail',
  standalone: true,
  imports: [RouterLink],
  template: `
    <div class="container my-5">
      <!-- Botón de retorno -->
      <a routerLink="/products" class="btn btn-outline-secondary mb-4">
        &larr; Volver a la lista
      </a>

      <!-- Manejo de estados del recurso individual -->
      @if (productResource.isLoading()) {
        <div class="card p-5 text-center shadow-sm">
```

```

        <div class="spinner-border text-primary mx-auto mb-3"
role="status"></div>
        <p class="text-muted mb-0">Cargando detalles del producto
#{{ productId() }}...</p>
    </div>
    } @else if (productResource.error()) {
        <div class="alert alert-danger shadow-sm">
            <h4 class="alert-heading">Error al cargar el producto</h4>
            <p>No se pudo obtener la información. Verifique que el ID
#{{ productId() }} sea correcto.</p>
        </div>
    } @else if (productResource.value(); as product) {
        <div class="card shadow-sm border-0 overflow-hidden">
            <div class="row g-0">
                <!-- Imagen del producto -->
                <div class="col-md-5 bg-light p-4 d-flex align-items-center justify-
content-center">
                    <img
                        [src]="product.image || 'https://via.placeholder.com/300'"
                        [alt]="product.title"
                        class="img-fluid rounded max-h-300"
                        style="max-height: 300px; object-fit: contain;"
                    />
                </div>

                <!-- Información detallada -->
                <div class="col-md-7 p-4 d-flex flex-column justify-content-
between">
                    <div>
                        <div class="d-flex justify-content-between align-items-start mb-
2">
                            <span class="badge bg-primary text-
uppercase">{{ product.category }}</span>
                            <span class="text-muted small">ID: #{{ product.id }}</span>
                        </div>
                        <h2 class="card-title fw-bold text-dark mb-
3">{{ product.title }}</h2>
                        <h3 class="text-success fw-bold mb-4">\${{ product.price }}</h3>
                        <p class="card-text text-secondary lh-
base">{{ product.description }}</p>
                    </div>

                    <div class="pt-3 border-top d-flex gap-2">
                        <button class="btn btn-success flex-grow-1">Comprar
ahora</button>
                        <a routerLink="/products" class="btn btn-light
border">Cerrar</a>
                    </div>
                </div>
            </div>
        </div>
    }
</div>
))
export class ProductDetailComponent {
    // Extrae los parámetros de la URL como Signals nativos
    private params = injectParams();

```

```

// Signal derivado con el ID de la ruta
productId = () => this.params()['id'];

// httpResource que reacciona automáticamente cuando cambia el productId()
productResource = httpResource<Product>(() => {
  const id = this.productId();
  return `https://fakestoreapi.com/products/${id}`;
});
}

```

Paso 2: Configurar las Rutas en `app.routes.ts`

Añadimos las rutas para navegar entre la lista principal y la vista de detalles del producto.

TypeScript

```

// src/app/app.routes.ts
import { Routes } from '@angular/router';

export const routes: Routes = [
  { path: '', redirectTo: 'products', pathMatch: 'full' },
  {
    path: 'products',
    loadComponent: () => import('./components/product-list/product-
list.component').then(m => m.ProductListComponent)
  },
  {
    path: 'products/:id',
    loadComponent: () => import('./components/product-detail/product-
detail.component').then(m => m.ProductDetailComponent)
  },
  { path: '**', redirectTo: 'products' }
];

```

Paso 3: Enlazar la Fila de la Tabla a la Vista de Detalles

Actualizamos la tabla del componente `product-list.component.ts` agregando un botón o enlace de "Ver detalles" utilizando la directiva `RouterLink`:

TypeScript

```

// En la columna de acciones dentro del template de product-list.component.ts:
<td class="text-end">
  <a
    [routerLink]="['/products', product.id]"
    class="btn btn-sm btn-outline-info me-2"
  >
    Ver
  </a>
  <button
    class="btn btn-sm btn-outline-warning me-2"
    (click)="openEditModal(product)"
  >
    Editar
  </button>
  <button
    class="btn btn-sm btn-outline-danger"
    (click)="deleteProduct(product.id)"
  >

```

```

      Eliminar
    </button>
  </td>

```

falta el router, no?

Tienes toda la razón, falta habilitar el enrutador a nivel global en la configuración de la aplicación y montar la etiqueta `<router-outlet />` en el componente principal.

Aquí tienes los dos archivos de configuración que faltan para habilitar la navegación:

1. Configurar los proveedores del Router (app.config.ts)

Debemos registrar `provideRouter` junto con las opciones de enrutamiento (como `withComponentInputBinding` para mapear parámetros directamente si fuera necesario):

TypeScript

```

// src/app/app.config.ts
import { ApplicationConfig, provideZoneChangeDetection } from '@angular/core';
import { provideRouter, withComponentInputBinding } from '@angular/router';
import { provideHttpClient, withFetch } from '@angular/common/http';
import { routes } from './app.routes';

export const appConfig: ApplicationConfig = {
  providers: [
    provideZoneChangeDetection({ eventCoalescing: true }),
    provideRouter(routes, withComponentInputBinding()),
    provideHttpClient(withFetch())
  ]
};

```

2. Montar el Outlet en el Componente Raíz (app.component.ts)

En el componente principal de la aplicación, importamos `RouterOutlet` y colocamos el tag `<router-outlet />` en la plantilla:

TypeScript

```

// src/app/app.component.ts
import { Component } from '@angular/core';
import { RouterOutlet } from '@angular/router';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [RouterOutlet],
  template: `
    <!-- Barra de navegación opcional -->
    <nav class="navbar navbar-dark bg-dark mb-4">
      <div class="container">
        <span class="navbar-brand mb-0 h1">Angular 22 CRUD App</span>
      </div>
    </nav>

    <!-- Punto de renderizado de las rutas (/products, /products/:id) -->
  `
})

```

```

    <main>
      <router-outlet />
    </main>
  )
}
export class AppComponent {}

```

Con `provideRouter(routes)` en `app.config.ts` y `<router-outlet />` en `app.component.ts`, la navegación entre la lista y la página de detalles queda totalmente funcional.

y donde esta el `app.routes`?

Aquí tienes el archivo `app.routes.ts` completo para conectar la lista de productos y la página de detalles con sus respectivas rutas:

TypeScript

```

// src/app/app.routes.ts
import { Routes } from '@angular/router';

export const routes: Routes = [
  // Ruta por defecto: redirige a la lista de productos
  {
    path: '',
    redirectTo: 'products',
    pathMatch: 'full'
  },

  // Ruta principal: Lista de productos (con Lazy Loading)
  {
    path: 'products',
    loadChildren: () =>
      import('./components/product-list/product-list.component')
        .then(m => m.ProductListComponent)
  },

  // Ruta de detalle: Carga la información de un producto por ID
  {
    path: 'products/:id',
    loadChildren: () =>
      import('./components/product-detail/product-detail.component')
        .then(m => m.ProductDetailComponent)
  },

  // Comodín: redirige cualquier ruta no encontrada a la lista
  {
    path: '**',
    redirectTo: 'products'
  }
];

```

Estructura final del proyecto

Para verificar que todos los bloques encajen correctamente:

Plaintext

```

src/
├── app/
│   ├── components/
│   │   ├── product-list/
│   │   │   └── product-list.component.ts
│   │   ├── product-detail/
│   │   │   └── product-detail.component.ts
│   │   └── product-form/
│   │       └── product-form.component.ts
│   ├── models/
│   │   └── product.model.ts
│   ├── services/
│   │   └── product.service.ts
│   ├── app.component.ts      <-- Contiene <router-outlet />
│   ├── app.config.ts        <-- Registra provideRouter(routes)
│   └── app.routes.ts         <-- Define las rutas (este archivo)
└── main.ts

```

creame tambien un search en el listado

Para añadir la búsqueda reactiva en la lista de productos, aprovecharemos el modelo de Signals de Angular 22 integrando el campo de texto directamente con el servicio y `httpResource`.

Paso 1: Actualizar el Servicio (`product.service.ts`)

Añadimos un Signal para el término de búsqueda (`searchTerm`) y actualizamos `httpResource` para reaccionar tanto a la categoría como al texto buscado.

TypeScript

```

// src/app/services/product.service.ts
import { Injectable, signal, httpResource, inject, computed } from
 '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Product, CreateProductDto } from '../models/product.model';

@Injectable({
  providedIn: 'root'
})
export class ProductService {
  private http = inject(HttpClient);
  private readonly apiUrl = 'https://fakestoreapi.com/products';

  // Signals para los filtros
  selectedCategory = signal<string>('');
  searchTerm = signal<string>(''); // <-- Nuevo Signal para el buscador

  // Resource base que obtiene los productos por categoría
  private baseProductsResource = httpResource<Product[]>(() => {
    const category = this.selectedCategory();
    return category
      ? `${this.apiUrl}/category/${category}`
      : this.apiUrl;
  });

  // Signal computado que filtra localmente por título en tiempo real
  products = computed(() => {

```



```

const list = this.baseProductsResource.value() ?? [];
const term = this.searchTerm().toLowerCase().trim();

if (!term) return list;

return list.filter(p => p.title.toLowerCase().includes(term));
});

// Exponemos los estados de carga y error del recurso base
isLoading = this.baseProductsResource.isLoading;
error = this.baseProductsResource.error;

reload() {
  this.baseProductsResource.reload();
}

// Operaciones CRUD
createProduct(product: CreateProductDto) {
  return this.http.post<Product>(this.apiUrl, product);
}

updateProduct(id: number, product: Partial<CreateProductDto>) {
  return this.http.put<Product>(`${this.apiUrl}/${id}`, product);
}

deleteProduct(id: number) {
  return this.http.delete<Product>(`${this.apiUrl}/${id}`);
}
}

```

Paso 2: Actualizar la Vista del Listado (product-list.component.ts)

Añadimos un campo de texto `<input>` que actualiza el Signal `searchTerm` en cada pulsación de tecla y reemplazamos la iteración de la tabla para consumir `productService.products()`.

TypeScript

```

// src/app/components/product-list/product-list.component.ts
import { Component, inject, signal } from '@angular/core';
import { RouterLink } from '@angular/router';
import { ProductService } from '../../../services/product.service';
import { Product } from '../../../models/product.model';
import { ProductFormComponent } from '../../product-form/product-form.component';

@Component({
  selector: 'app-product-list',
  standalone: true,
  imports: [ProductFormComponent, RouterLink],
  template: `
    <div class="container my-4">
      <div class="d-flex justify-content-between align-items-center mb-4">
        <h2>Gestión de Productos</h2>
        <button class="btn btn-primary" (click)="openCreateModal()">
          + Nuevo Producto
        </button>
      </div>
    </div>
  `
})

```

```

<!-- Barra de Filtros: Búsqueda y Categoría -->
<div class="row g-3 mb-4">

  <!-- Campo de Búsqueda -->
  <div class="col-md-8">
    <label class="form-label fw-bold">Buscar por nombre:</label>
    <div class="input-group">
      <span class="input-group-text bg-white"> </span>
      <input
        type="text"
        class="form-control"
        placeholder="Escribe para buscar..."
        [value]="productService.searchTerm()"
        (input)="onSearchInput($event)"
      />
      @if (productService.searchTerm()) {
        <button
          class="btn btn-outline-secondary"
          type="button"
          (click)="clearSearch()"
        >
          x
        </button>
      }
    </div>
  </div>

  <!-- Filtro por Categoría -->
  <div class="col-md-4">
    <label class="form-label fw-bold">Filtrar por categoría:</label>
    <select
      class="form-select"
      (change)="onCategoryChange($event)"
    >
      <option value="">Todas las categorías</option>
      <option value="electronics">Electrónica</option>
      <option value="jewelery">Joyería</option>
      <option value="men's clothing">Ropa de Hombre</option>
    </select>
  </div>

</div>

<!-- Manejo de Estados con httpResource -->
@if (productService.isLoading()) {
  <div class="alert alert-info d-flex align-items-center gap-2">
    <div class="spinner-border spinner-border-sm" role="status"></div>
    <span>Cargando productos...</span>
  </div>
} @else if (productService.error()) {
  <div class="alert alert-danger">
    Ocurrió un error al cargar la lista: {{ productService.error() }}
  </div>
} @else {
  <table class="table table-hover border align-middle">
    <thead class="table-light">
      <tr>
        <th>ID</th>
        <th>Título</th>

```

```

        <th>Precio</th>
        <th>Categoría</th>
        <th class="text-end">Acciones</th>
    </tr>
</thead>
<tbody>
    <!-- Iteramos sobre el Signal computado productService.products() --
>
    @for (product of productService.products(); track product.id) {
        <tr>
            <td>{{ product.id }}</td>
            <td class="fw-medium">{{ product.title }}</td>
            <td>\${{ product.price }}</td>
            <td><span class="badge bg-secondary">{{ product.category
}}</span></td>
            <td class="text-end">
                <a
                    [routerLink]="['/products', product.id]"
                    class="btn btn-sm btn-outline-info me-2"
                >
                    Ver
                </a>
                <button
                    class="btn btn-sm btn-outline-warning me-2"
                    (click)="openEditModal(product)"
                >
                    Editar
                </button>
                <button
                    class="btn btn-sm btn-outline-danger"
                    (click)="deleteProduct(product.id)"
                >
                    Eliminar
                </button>
            </td>
        </tr>
    } @empty {
        <tr>
            <td colspan="5" class="text-center text-muted py-4">
                No se encontraron productos que coincidan con la búsqueda.
            </td>
        </tr>
    }
</tbody>
</table>
}

<!-- Formulario Modal -->
@if (showFormModal()) {
    <app-product-form
        [productToEdit]="selectedProduct()"
        (closeModal)="closeFormModal()"
    />
}
</div>
})
export class ProductListComponent {
    productService = inject(ProductService);

```

```

showFormModal = signal(false);
selectedProduct = signal<Product | null>(null);

onSearchInput(event: Event) {
  const value = (event.target as HTMLInputElement).value;
  this.productService.searchTerm.set(value);
}

clearSearch() {
  this.productService.searchTerm.set('');
}

onCategoryChange(event: Event) {
  const value = (event.target as HTMLSelectElement).value;
  this.productService.selectedCategory.set(value);
}

openCreateModal() {
  this.selectedProduct.set(null);
  this.showFormModal.set(true);
}

openEditModal(product: Product) {
  this.selectedProduct.set(product);
  this.showFormModal.set(true);
}

closeFormModal() {
  this.showFormModal.set(false);
}

deleteProduct(id: number) {
  if (confirm('¿Estás seguro de eliminar este producto?')) {
    this.productService.deleteProduct(id).subscribe({
      next: () => {
        alert('Producto eliminado correctamente');
        this.productService.reload();
      }
    });
  }
}
}

```

Beneficios de esta implementación:

- Sin RxJS ni debounceTime manual: El uso de computed() realiza el filtrado en memoria de manera inmediata sin causar re-renderizados innecesarios del DOM fuera de las filas afectadas.
- Integración limpia: El botón "X" limpia el Signal searchTerm(), refrescando instantáneamente la tabla.

que significa esto: "'@angular/core'" has no exported member named 'httpResource'. Did you mean 'Resource'?ts(2724)

```
import httpResource
```

Este error de TypeScript ocurre porque `httpResource` no forma parte de `@angular/core`.

En Angular (desde su introducción experimental en v19 hasta su estabilización en v22), las utilidades HTTP basadas en Signals se exportan desde el paquete `@angular/common/http`, mientras que `@angular/core` solo contiene la función genérica `resource()`.

Cómo solucionarlo

Cambia la importación en tu archivo para traer `httpResource` desde `@angular/common/http`:

Incorrecto (Genera el error `ts(2724)`):

TypeScript

```
import { Component, signal, httpResource } from '@angular/core'; //
```

Correcto:

TypeScript

```
import { Component, signal } from '@angular/core';
import { httpResource } from '@angular/common/http'; //
```

Resumen de paquetes de APIs de Signals:

esta línea da errores: `import { signalForm, formField, validators } from '@angular/forms/signals';`
`Module '"@angular/forms/signals"' has no exported member 'signalForm'.ts(2305)`

```
import signalForm
```

El error `ts(2305)` sucede porque `signalForm` y `formField` no existen como nombres de importación en la API final de Signal Forms (`@angular/forms/signals`).

En la arquitectura de Signal Forms, no creas los campos manualmente uno por uno. En su lugar:

1. Creas un Signal con el modelo de datos.
2. Usas la función `form()` para generar la estructura del formulario.
3. Importas la directiva `FormField` (con Mayúscula) para enlazar el HTML con `[formField]`.

GitHub

4. Las reglas de validación (`required`, `min`, `email`, `debounce`) se importan directamente desde `@angular/forms/signals`.

GitHub

1. Solución: Importaciones Correctas

Sustituye la línea con error por las importaciones oficiales:

TypeScript

```
// INCORRECTO:
// import { signalForm, formField, validators } from '@angular/forms/signals';

// CORRECTO:
import { form, FormField, required, min, debounce } from
 '@angular/forms/signals';
```

2. Ajuste en el Componente (product-form.component.ts)

Así debe quedar la inicialización del formulario en tu componente:

TypeScript

```
import { Component, inject, input, output, signal, OnInit } from
 '@angular/core';
// Importaciones de Signal Forms
import { form, FormField, required, min, debounce } from
 '@angular/forms/signals';
import { ProductService } from '../services/product.service';
import { Product } from '../models/product.model';

@Component({
  selector: 'app-product-form',
  standalone: true,
  imports: [FormField], // <-- Es indispensable importar la directiva FormField
  template: `
    <div class="modal d-block bg-dark bg-opacity-50" tabindex="-1">
      <div class="modal-dialog modal-dialog-centered">
        <div class="modal-content">
          <div class="modal-header">
            <h5 class="modal-title">
              {{ productToEdit() ? 'Editar Producto' : 'Nuevo Producto' }}
            </h5>
            <button type="button" class="btn-close"
(click)="closeModal.emit()"></button>
          </div>

          <form (submit)="onSubmit($event)">
            <div class="modal-body">

              <!-- Título -->
              <div class="mb-3">
                <label class="form-label">Título</label>
                <!-- Acceso a la estructura de la propiedad title -->
                <input
                  type="text"
                  class="form-control"
                  [formField]="productForm.title"
                  placeholder="Ej. Teclado Mecánico"
                />
                <!-- Para leer banderas como invalid/errors se invoca la
propiedad como Signal: .title() -->
                @if (productForm.title().invalid() &&
productForm.title().touched()) {
                  <small class="text-danger d-block mt-1">El título es
obligatorio.</small>
                }
              </div>
            </div>
          </form>
        </div>
      </div>
    </div>
  `
})
```

```

</div>

<!-- Precio -->
<div class="mb-3">
  <label class="form-label">Precio</label>
  <input
    type="number"
    class="form-control"
    [formField]="productForm.price"
  />
  @if (productForm.price().invalid() &&
productForm.price().touched()) {
    <small class="text-danger d-block mt-1">El precio debe ser
mayor a 0.</small>
  }
</div>

<!-- Categoría -->
<div class="mb-3">
  <label class="form-label">Categoría</label>
  <select class="form-select" [formField]="productForm.category">
    <option value="">Selecciona una opción</option>
    <option value="electronics">Electrónica</option>
    <option value="jewelery">Joyería</option>
    <option value="men's clothing">Ropa de Hombre</option>
  </select>
</div>

<!-- Descripción -->
<div class="mb-3">
  <label class="form-label">Descripción</label>
  <textarea
    class="form-control"
    rows="3"
    [formField]="productForm.description"
  ></textarea>
</div>

</div>

<div class="modal-footer">
  <button
    type="button"
    class="btn btn-secondary"
    (click)="closeModal.emit()"
  >
    Cancelar
  </button>
  <button
    type="submit"
    class="btn btn-success"
    [disabled]="productForm().invalid()"
  >
    Guardar
  </button>
</div>
</form>
</div>
</div>

```

```

    </div>
  )
}
export class ProductFormComponent implements OnInit {
  private productService = inject(ProductService);

  productToEdit = input<Product | null>(null);
  closeModal = output<void>();

  // 1. Modelo de datos en un Signal fuente
  productModel = signal({
    title: '',
    price: 0,
    category: 'electronics',
    description: ''
  });

  // 2. Creación del formulario usando la función form() y su esquema
  productForm = form(this.productModel, (s) => {
    required(s.title, { message: 'El título es obligatorio' });
    debounce(s.title, 250);

    required(s.price);
    min(s.price, 0.01, { message: 'El precio debe ser mayor a 0' });

    required(s.category);
  });

  ngOnInit() {
    const product = this.productToEdit();
    if (product) {
      // Para cargar valores en Signal Forms actualizamos el Signal fuente
      this.productModel.set({
        title: product.title,
        price: product.price,
        category: product.category,
        description: product.description
      });
    }
  }

  onSubmit(event: Event) {
    event.preventDefault();

    // Comprobación de validez llamando al formulario como Signal: productForm()
    if (this.productForm().invalid()) return;

    // Los datos del formulario provienen directamente del Signal fuente
    const formValues = this.productModel();
    const productData = {
      title: formValues.title,
      price: Number(formValues.price),
      category: formValues.category,
      description: formValues.description
    };

    const currentProduct = this.productToEdit();

    if (currentProduct) {

```



```

        this.productService.updateProduct(currentProduct.id,
productData).subscribe({
            next: () => {
                this.productService.reload();
                this.closeModal.emit();
            }
        });
    } else {
        this.productService.createProduct(productData).subscribe({
            next: () => {
                this.productService.reload();
                this.closeModal.emit();
            }
        });
    }
}
}
}

```

Resumen de diferencias de la sintaxis oficial:

me da un error con injectParams

El error con `injectParams` se debe a que no es una función nativa de `@angular/router`, sino un helper de la librería comunitaria `ngxtension` o una función personalizada.

`ngxtension`

Existen dos formas estándar y limpias de resolver este problema en Angular:

Opción 1 (Recomendada): Usar Component Inputs con `withComponentInputBinding()`

Como ya registramos `withComponentInputBinding()` en el `app.config.ts`, el enrutador de Angular inyecta los parámetros de la URL directamente como un `Signal input()`.

Angular

En `product-detail.component.ts`:

TypeScript

```

import { Component, input, httpResource } from '@angular/core';
import { RouterLink } from '@angular/router';
import { Product } from '../models/product.model';

@Component({
  selector: 'app-product-detail',
  standalone: true,
  imports: [RouterLink],
  template: `
    <div class="container my-5">
      <a routerLink="/products" class="btn btn-outline-secondary mb-4">&larr;
Volver</a>

      @if (productResource.isLoading()) {

```

```

        <div class="alert alert-info">Cargando producto #{{ id() }}...</div>
    } @else if (productResource.error()) {
        <div class="alert alert-danger">Error al cargar el producto.</div>
    } @else if (productResource.value(); as product) {
        <div class="card p-4">
            <h2>{{ product.title }}</h2>
            <h4 class="text-success">\${{ product.price }}</h4>
            <p>{{ product.description }}</p>
        </div>
    }
</div>
,
})
export class ProductDetailComponent {
    // Angular mapea automáticamente :id de la ruta a este Signal input
    id = input.required<string>();

    // httpResource detecta cuando id() cambia y realiza la consulta
    automáticamente
    productResource = httpResource<Product>(() =>
        `https://fakestoreapi.com/products/${this.id()}`
    );
}

```

Opción 2: Usar toSignal con ActivatedRoute (Sin librerías externas)

Si prefieres obtener el parámetro usando las APIs nativas de @angular/router y convertirlas a Signal:

TypeScript

```

import { Component, inject, httpResource } from '@angular/core';
import { ActivatedRoute, RouterLink } from '@angular/router';
import { toSignal } from '@angular/core/rxjs-interop';
import { map } from 'rxjs/operators';
import { Product } from '../../models/product.model';

@Component({
    selector: 'app-product-detail',
    standalone: true,
    imports: [RouterLink],
    template: `...`
})
export class ProductDetailComponent {
    private route = inject(ActivatedRoute);

    // Convertimos los parámetros del ActivatedRoute a un Signal
    productId = toSignal(
        this.route.params.pipe(map(params => params['id']))
    );

    productResource = httpResource<Product>(() => {
        const id = this.productId();
        return id ? `https://fakestoreapi.com/products/${id}` : undefined;
    });
}

```

Opción 3: Instalar `ngxtension` (Si quieres usar `injectParams`)

Si deseas usar la sintaxis exacta de `injectParams()`, debes instalar el paquete de utilidad `ngxtension` e importarlo:

`ngxtension`

Bash

```
npm install ngxtension
```

Y en tu componente:

TypeScript

```
import { injectParams } from 'ngxtension/inject-params';

export class ProductDetailComponent {
  // Extrae el parámetro 'id' como un Signal nativo
  productId = injectParams('id');
}
```

otro error: Property 'image' does not exist on type 'Product'.ngtsc(2339)

product-detail.c.component.ts(9, 16): Error occurs in the template of component ProductDetailCComponent.

any

Este error ocurre porque en la interfaz `Product` (`src/app/models/product.model.ts`) no declaramos la propiedad opcional `image`.

Para corregirlo, simplemente añade la propiedad `image` a tu archivo de modelo.

Paso 1: Actualizar `src/app/models/product.model.ts`

Añade `image?: string` a la interfaz:

TypeScript

```
// src/app/models/product.model.ts
export interface Product {
  id: number;
  title: string;
  price: number;
  category: string;
  description: string;
  image?: string; // <-- Propiedad añadida
}

export type CreateProductDto = Omit<Product, 'id'>;
```

Paso 2: Alternativa rápida en la Plantilla

Si no deseas modificar el modelo por ahora, puedes evitar el chequeo de tipo estricto en la plantilla accediendo a la propiedad mediante corchetes o un cast seguro:

HTML

```
<!-- Opción sin modificar la interfaz -->
<img
  [src]="$any(product).image || 'https://via.placeholder.com/300'"
  [alt]="product.title"
  class="img-fluid rounded"
/>
```

(La solución recomendada y más limpia para mantener el tipado estricto es agregar la propiedad directamente en el modelo `product.model.ts`).